IDMA 2024 Problem set 4

Lauritz Tørholm Jørgensen, fqm630@alumni.ku.dk

June 2026

1

1a

In Figure 1a there are 7 layers. The walk starts at the single vertex in the first layer and makes one choice when moving from one layer to the next. Hence the walk makes

$$7 - 1 = 6$$

choices in total.

At each step the walk either goes left or right. All choices are equally likely, so all possible walks have probability

$$\frac{1}{2^6} = \frac{1}{64}.$$

To end in the sink s_j, the walk must have gone right exactly j times. This is because going left keeps the vertex number the same, while going right increases it by 1.

Thus the number of walks ending in s_j is the number of ways to choose which j of the 6 steps are right steps. This is

$$\binom{6}{j}.$$

Therefore

$$\Pr[\text{walk ends in } s_j] = \frac{\binom{6}{j}}{2^6}.$$

For the sinks in Figure 1a this gives

$$\Pr[s_0] = \frac{\binom{6}{0}}{64} = \frac{1}{64},$$

$$\Pr[s_1] = \frac{\binom{6}{1}}{64} = \frac{6}{64} = \frac{3}{32},$$

$$\Pr[s_2] = \frac{\binom{6}{2}}{64} = \frac{15}{64},$$

$$\Pr[s_3] = \frac{\binom{6}{3}}{64} = \frac{20}{64} = \frac{5}{16},$$

$$\Pr[s_4] = \frac{\binom{6}{4}}{64} = \frac{15}{64},$$

$$\Pr[s_5] = \frac{\binom{6}{5}}{64} = \frac{6}{64} = \frac{3}{32},$$

and

$$\Pr[s_6] = \frac{\binom{6}{6}}{64} = \frac{1}{64}.$$

The largest probability is

$$\frac{20}{64} = \frac{5}{16},$$

so the walk is most likely to end in s_3.

1b

Now consider a lattice graph with L layers.

The walk starts in layer 0 and ends in layer $L-1$. Therefore it makes

$$L-1$$

steps.

At each step there are two choices, left or right. Thus the total number of possible walks is

$$2^{L-1}.$$

All of these walks are equally likely, since each coin flip has probability 1/2.

As before, going left does not change the vertex number, while going right increases the vertex number by 1. Therefore, to end in vertex j in layer $L-1$, the walk must go right exactly j times.

The number of walks with exactly j right steps among the $L-1$ steps is

$$\binom{L-1}{j}.$$

Hence the probability that the walk ends in vertex j in layer $L-1$ is

$$\Pr[\text{walk ends in vertex } j] = \frac{\binom{L-1}{j}}{2^{L-1}},$$

for

$$j = 0, 1, \ldots, L-1.$$

This is just the binomial distribution with $L-1$ independent coin flips, where j counts the number of right moves.

2

2a

We run Prim's algorithm starting at vertex a.

The initial key values are

$$key(a) = 0$$

and

$$key(b) = key(c) = key(d) = key(e) = key(f) = \infty.$$

The initial heap array is in lexicographic order:

$$[(a, 0), (b, \infty), (c, \infty), (d, \infty), (e, \infty), (f, \infty)].$$

I assume that equal keys are not swapped in the heap.

First we dequeue a. After removing a, the last element f is moved to the root. Since all remaining keys are ∞, no swap is needed. The heap becomes

$$[(f, \infty), (b, \infty), (c, \infty), (d, \infty), (e, \infty)].$$

Now we consider the neighbours of a in lexicographic order.

First b is considered. The edge (a, b) has weight 4, and since

$$4 < \infty,$$

we set

$$key(b) = 4, \qquad \pi(b) = a.$$

After the decrease-key operation, b moves to the root:

$$[(b, 4), (f, \infty), (c, \infty), (d, \infty), (e, \infty)].$$

Next c is considered. The edge (a, c) has weight 1, and since

$$1 < \infty,$$

we set

$$key(c) = 1, \qquad \pi(c) = a.$$

The heap becomes

$$[(c, 1), (f, \infty), (b, 4), (d, \infty), (e, \infty)].$$

Finally d is considered. The edge (a, d) has weight 5, and since

$$5 < \infty,$$

we set

$$key(d) = 5, \qquad \pi(d) = a.$$

After the decrease-key operation, the heap is

$$[(c, 1), (d, 5), (b, 4), (f, \infty), (e, \infty)].$$

The next vertex dequeued is c. After removing c, the last element e is moved to the root:

$$[(e, \infty), (d, 5), (b, 4), (f, \infty)].$$

Heapify swaps e with b, since b has the smallest key among the children. The heap after the dequeue operation is therefore

$$[(b, 4), (d, 5), (e, \infty), (f, \infty)].$$

Now we consider the neighbours of c in lexicographic order.

First a is considered, but a has already been removed from the heap, so we ignore it.

Next b is considered. The edge (c, b) has weight 2, and since

$$2 < key(b) = 4,$$

we update

$$key(b) = 2, \qquad \pi(b) = c.$$

Since b is already at the root, the heap stays

$$[(b, 2), (d, 5), (e, \infty), (f, \infty)].$$

Next d is considered. The edge (c, d) has weight 6, but

$$6 > key(d) = 5,$$

so we do not update d.

Finally e is considered. The edge (c, e) has weight 4, and since

$$4 < \infty,$$

we update

$$key(e) = 4, \qquad \pi(e) = c.$$

The heap is now

$$[(b, 2), (d, 5), (e, 4), (f, \infty)].$$

The next vertex dequeued is b. The edge added to the tree is therefore (c, b).

The neighbours of b are a, c, and d. The vertices a and c have already been removed from the heap, so they are ignored. For d, the edge (b, d) has weight 3, and

$$3 < key(d) = 5.$$

So we update

$$key(d) = 3, \qquad \pi(d) = b.$$

The next vertex dequeued is d. The edge added to the tree is (b, d).

The neighbours of d are a, b, c, e, and f. The vertices a, b, and c have already been removed from the heap, so they are ignored.

For e, the edge (d, e) has weight 5, but

$$5 > key(e) = 4,$$

so e is not updated.

For f, the edge (d, f) has weight 1, and

$$1 < key(f) = \infty.$$

So we update

$$key(f) = 1, \qquad \pi(f) = d.$$

The next vertex dequeued is f. The edge added to the tree is (d, f).

The neighbours of f are d and e. The vertex d has already been removed from the heap, so it is ignored. For e, the edge (f, e) has weight 3, and

$$3 < key(e) = 4.$$

So we update

$$key(e) = 3, \qquad \pi(e) = f.$$

The last vertex dequeued is e. The edge added to the tree is (f, e).

Thus the minimum spanning tree produced by Prim's algorithm is

$$T = \{(a, c), (c, b), (b, d), (d, f), (f, e)\}.$$

The total weight is

$$1 + 2 + 3 + 1 + 3 = 10.$$

2b

Jakob's algorithm is not correct.

We can see this already on the graph from Figure 2. Suppose that the randomly chosen start vertex is d.

From d, the cheapest edge to a vertex not in S is

$$(d, f)$$

with weight 1.

From f, the cheapest edge to a vertex not in S is

$$(f, e)$$

with weight 3.

From e, the only remaining unvisited neighbour is c, so the algorithm chooses

$$(e, c)$$

with weight 4.

From c, the cheapest edge to a vertex not in S is

$$(c, a)$$

with weight 1.

From a, the only remaining unvisited neighbour is b, so the algorithm chooses

$$(a, b)$$

with weight 4.

Thus Jakob's algorithm can return the tree

$$T' = \{(d, f), (f, e), (e, c), (c, a), (a, b)\}.$$

The weight of this tree is

$$1 + 3 + 4 + 1 + 4 = 13.$$

But in part 2a we found a spanning tree of weight

$$10.$$

Therefore T' is not a minimum spanning tree. This gives a counterexample, so Jakob's algorithm is not a correct MST algorithm.

Jakob's time complexity analysis is also not correct as stated.

The expensive part is the step where the algorithm has to choose the cheapest edge from the current vertex w to a vertex not yet in S. If the adjacency lists are not sorted by edge weight, then this may require scanning the whole adjacency list of w.

The same vertex can become the current vertex many times because of the backtracking step. For example, consider a star graph with center vertex r and $n-1$ leaves. If the algorithm starts at r, then after visiting each leaf it has to return to r to choose another leaf. If it scans the whole adjacency list of r each time, then it spends

$$\Theta(n)$$

time for each of the

$$n-1$$

leaves. This gives total time

$$\Theta(n^2).$$

But in this star graph,

$$|V| = n$$

and

$$|E| = n-1,$$

so

$$|V| + |E| = \Theta(n).$$

Thus the algorithm is not necessarily $O(|V| + |E|)$ with ordinary adjacency lists.

If the adjacency lists were already sorted by edge weight and we kept suitable pointers, then the traversal part could be made linear. But this is an extra assumption. As stated, Jakob has not justified the claimed running time.

3

3a

From Figure 3, the adjacency lists, in lexicographic order, are

$$\begin{array}{rcl} a & : & b, d, e, g, \\ b & : & a, c, e, f, \\ c & : & b, \\ d & : & a, h, i, \\ e & : & a, b, f, \\ f & : & b, e, \\ g & : & a, h, \\ h & : & d, g, i, \\ i & : & d, h. \end{array}$$

We run depth-first search starting at a.

First we visit a. The neighbours of a are considered in the order

$$b, d, e, g.$$

The first neighbour is b, which has not been visited, so we add the tree edge

$$(a, b)$$

and recursively visit b.

At b, the neighbours are

$$a, c, e, f.$$

The vertex a has already been visited, so it is ignored. The next neighbour is c, which has not been visited, so we add

$$(b, c)$$

and recursively visit c.

At c, the only neighbour is b. This has already been visited, so the recursive call on c ends.

We return to b. The next neighbour of b is e, which has not been visited. Hence we add

$$(b, e)$$

and recursively visit e.

At e, the neighbours are

$$a, b, f.$$

The vertices a and b have already been visited. The next neighbour is f, which has not been visited, so we add

$$(e, f)$$

and recursively visit f.

At f, the neighbours are

$$b, e.$$

Both have already been visited, so the recursive call on f ends. Then the recursive call on e also ends.

We return to b. Its last neighbour is f, which has already been visited, so the recursive call on b ends.

We return to a. The next neighbour of a is d, which has not been visited. Hence we add

$$(a, d)$$

and recursively visit d.
At d, the neighbours are

$$a, h, i.$$

The vertex a has already been visited. The next neighbour is h, which has not been visited, so we add

$$(d, h)$$

and recursively visit h.
At h, the neighbours are

$$d, g, i.$$

The vertex d has already been visited. The next neighbour is g, which has not been visited, so we add

$$(h, g)$$

and recursively visit g.
At g, the neighbours are

$$a, h.$$

Both have already been visited, so the recursive call on g ends.
We return to h. The next neighbour of h is i, which has not been visited. Hence we add

$$(h, i)$$

and recursively visit i.
At i, the neighbours are

$$d, h.$$

Both have already been visited, so the recursive call on i ends. Then the recursive call on h ends.
We return to d. Its last neighbour is i, which has already been visited, so the recursive call on d ends.
Finally, we return to a. The remaining neighbours e and g have already been visited, so the search ends.
The vertices are first visited in the order

$$a, b, c, e, f, d, h, g, i.$$

The depth-first search tree is

$$\{(a, b), (b, c), (b, e), (e, f), (a, d), (d, h), (h, g), (h, i)\}.$$

3b

We now run breadth-first search starting at a.

Initially, only a has been discovered, and the queue is

$$[a].$$

We dequeue a. Its neighbours are

$$b, d, e, g.$$

All four are undiscovered, so we discover them in that order and add the tree edges

$$(a, b), \quad (a, d), \quad (a, e), \quad (a, g).$$

The queue is now

$$[b, d, e, g].$$

Next we dequeue b. Its neighbours are

$$a, c, e, f.$$

The vertex a has already been discovered. The vertex c is undiscovered, so we discover it and add

$$(b, c).$$

The vertex e has already been discovered. The vertex f is undiscovered, so we discover it and add

$$(b, f).$$

The queue is now

$$[d, e, g, c, f].$$

Next we dequeue d. Its neighbours are

$$a, h, i.$$

The vertex a has already been discovered. The vertices h and i are undiscovered, so we discover them and add

$$(d, h), \quad (d, i).$$

The queue is now

$$[e, g, c, f, h, i].$$

Next we dequeue e. Its neighbours are

$$a, b, f.$$

All three have already been discovered, so nothing is added. The queue is

$$[g, c, f, h, i].$$

Next we dequeue g. Its neighbours are

$$a, h.$$

Both have already been discovered, so nothing is added. The queue is

$$[c, f, h, i].$$

Next we dequeue c. Its only neighbour is b, which has already been discovered. The queue is

$$[f, h, i].$$

Next we dequeue f. Its neighbours are

$$b, e.$$

Both have already been discovered, so nothing is added. The queue is

$$[h, i].$$

Next we dequeue h. Its neighbours are

$$d, g, i.$$

All three have already been discovered, so nothing is added. The queue is

$$[i].$$

Finally, we dequeue i. Its neighbours are

$$d, h.$$

Both have already been discovered, so nothing is added. The queue is empty, and the search ends.

The vertices are discovered in the order

$$a, b, d, e, g, c, f, h, i.$$

The breadth-first search tree is

$$\{(a, b), (a, d), (a, e), (a, g), (b, c), (b, f), (d, h), (d, i)\}.$$

3c

Now every edge has weight 1, and we run Dijkstra's algorithm starting at a.

Since all edge weights are equal to 1, Dijkstra's algorithm processes vertices by increasing distance from a. These distances are the same as the BFS layers.

The distances from a are

$$\text{dist}(a) = 0,$$

$$\text{dist}(b) = \text{dist}(d) = \text{dist}(e) = \text{dist}(g) = 1,$$

and

$$\text{dist}(c) = \text{dist}(f) = \text{dist}(h) = \text{dist}(i) = 2.$$

With the same lexicographic order for breaking ties, Dijkstra's algorithm gives the tree

$$\{(a,b),(a,d),(a,e),(a,g),(b,c),(b,f),(d,h),(d,i)\}.$$

This is the same tree as BFS in part 3b.

It is not the same as the DFS tree in part 3a. For example, in the DFS tree the vertex g is reached by the path

$$a, d, h, g,$$

even though g is adjacent to a. Similarly, f is reached by

$$a, b, e, f,$$

although f is at distance 2 from a. DFS does not necessarily produce shortest paths from the start vertex.

BFS does produce shortest paths in an unweighted graph. Dijkstra's algorithm with all edge weights equal to 1 also produces shortest paths. Therefore Dijkstra behaves like BFS in this case, up to possible differences caused only by tie-breaking.